



UNIVERSITY OF BUEA

Faculty of Engineering and Technology

Internet Programming & Mobile Programming

Task 6: Database Design and Implementation

Group 12

Course Lecturer: Dr. Nkemeni Valerie

Academic Year: 2025/2026

Group Members

Full Name	Matricule
AWA PRECIOUS NKENG	FE23A019
GUIMAZEU KENFACK DANIELLE	FE23A067
NGANE IDRIS AJANG	FE23A110
TACHINDA NJIGONG TRESOR	FE23A147
TAMBE ENOWTABI PRINCELY	FE23A155

Contents

Internet Programming & Mobile Programming.....	i
<i>Task 6: Database Design and Implementation</i>	i
Group 12	i
Group Members	ii
1. Introduction	1
2. Data Elements	1
2.1 User Data Elements	1
2.2 Fault Data Elements.....	1
2.3 Fault Embedding Data Elements.....	2
2.4 Diagnosis Data Elements	3
2.5 Video Data Elements	3
3. Conceptual Design	4
3.1 Entities.....	4
3.2 Relationships	4
3.3 Business Rules	4
4. Entity Relationship Diagram	5
5. Database Implementation.....	6
5.1 Database Management System	6
5.2 Enabling the Vector Extension	7
5.3 Table Creation.....	7
5.4 Vector Index for Fast Search.....	7
6. Backend Implementation	7
6.1 Layered Architecture.....	7
6.2 Entity Mapping	8
6.3 Repositories	8
6.4 The Diagnosis Pipeline.....	8
7. Connecting the Database to the Backend.....	8
7.1 Connection Configuration.....	8
7.2 Object Relational Mapping Configuration.....	9
7.3 How the Connection Works	9
7.4 Data Population.....	9
8. Conclusion.....	9

1. Introduction

After completing the requirement analysis, system design, and visual design phases, the next stage in the development of the CarDiag mobile application was the design and implementation of the database. The database forms the foundation of the entire system, as it stores the fault knowledge base, user accounts, diagnostic records, embedding vectors, and cached video tutorials.

CarDiag is an artificial intelligence based mobile application that helps vehicle owners identify car faults by scanning dashboard warning lights, recording engine sounds, or describing problems in text. The application matches these inputs against a database of over six thousand standardized OBD-II fault codes and returns an explanation, urgency level, repair tips, and relevant video tutorials.

This report describes the database design process for CarDiag, beginning with the identification of data elements, followed by the conceptual design, the entity relationship diagram, the physical database implementation, the backend implementation, and finally how the database is connected to the backend.

2. Data Elements

Data elements are the individual pieces of information that the system needs to store. They were identified directly from the functional requirements of the application. The following sub-sections describe the main data elements grouped by the entity they belong to.

2.1 User Data Elements

These store information about registered users of the application. A user can register, log in, and view their personal diagnosis history.

Data Element	Description	Data Type
id	Unique identifier for each user	Integer
firstName	User's first name	String
lastName	User's last name	String
email	Unique email used for login	String
password	Securely hashed password	String
role	User role (USER or ADMIN)	String
createdAt	Account creation date and time	Timestamp

2.2 Fault Data Elements

These store the catalogue of known vehicle faults. This is the core knowledge base against which all user inputs are matched.

Data Element	Description	Data Type
id	OBD-II fault code (e.g. B1200, P0301)	String
name	Name of the fault	String
category	Fault category (Engine, Brakes, etc.)	Enum
description	Full description of the fault	Text
symptoms	List of observable symptoms	Text (JSON)
causes	List of probable causes	Text (JSON)
repairTips	List of recommended repair actions	Text (JSON)
urgency	Urgency level (CRITICAL to LOW)	Enum
warningLight	Associated dashboard warning light	Enum
createdAt	Record creation date and time	Timestamp

2.3 Fault Embedding Data Elements

These store the numerical vector representation of each fault, used by the artificial intelligence component to perform similarity search.

Data Element	Description	Data Type
id	Unique identifier for each embedding	Integer
fault_id	Reference to the fault it belongs to	String (FK)
modality	Type of source (TEXT, IMAGE, AUDIO)	Enum
sourceText	The text that was embedded	Text
embedding	1024-dimensional vector	Vector (1024)
modelVersion	Embedding model used (voyage-3)	String
createdAt	Embedding creation date and time	Timestamp

2.4 Diagnosis Data Elements

These store the result of every diagnosis performed, forming the user's history and enabling future analysis.

Data Element	Description	Data Type
id	Unique identifier for each diagnosis	Integer
user_id	Reference to user (null for guests)	Integer (FK)
inputType	Type of input (IMAGE, AUDIO, TEXT)	Enum
imageDescription	Description of submitted image	Text
audioTranscription	Description of submitted audio	Text
userText	Text submitted by the user	Text
matched_fault_id	The fault that was matched	String (FK)
confidence	Confidence score of the match	Double
distance	Vector distance of the match	Double
llmReport	AI-generated diagnosis report	Text
createdAt	Diagnosis date and time	Timestamp

2.5 Video Data Elements

These store video tutorial links fetched per fault, cached to reduce calls to the external video service.

Data Element	Description	Data Type
id	Unique identifier for each cached video	Integer
fault_id	The fault the video relates to	String (FK)
videoId	YouTube video identifier	String
title	Title of the video	Text
channelName	Name of the YouTube channel	Text
thumbnailUrl	Link to the video thumbnail	Text
videoUrl	Full link to the video	Text
cachedAt	When the video was cached	Timestamp
expiresAt	When the cache entry expires	Timestamp

3. Conceptual Design

The conceptual design defines the main entities of the system and the relationships between them, without concern for how they will be physically stored. It provides a high-level view of the data structure.

3.1 Entities

Five main entities were identified for the CarDiag database:

- User: represents a registered user of the application.
- Fault: represents a known vehicle fault from the OBD-II catalogue.
- FaultEmbedding: represents the vector form of a fault, used for AI similarity search.
- Diagnosis: represents a single diagnosis performed by a user.
- Video: represents cached video tutorials linked to a fault.

3.2 Relationships

The relationships between the entities are as follows:

Relationship	Type	Description
User — Diagnosis	One-to-Many	A user can have many diagnoses; a diagnosis belongs to one user (or none for guests).
Fault — FaultEmbedding	One-to-One	Each fault has exactly one embedding vector.
Fault — Diagnosis	One-to-Many	A fault can be matched in many diagnoses; each diagnosis matches one fault.
Fault — Video	One-to-Many	A fault can have many cached videos; each cached video belongs to one fault.

3.3 Business Rules

The following business rules govern the data:

- Every fault must have a unique fault code as its identifier.
- A diagnosis may exist without a user, to support guest usage of the application.
- Each fault has exactly one embedding, enforced by a unique constraint.
- A cached video automatically expires after a set period to keep content fresh.
- Deleting a fault automatically removes its embedding and cached videos.

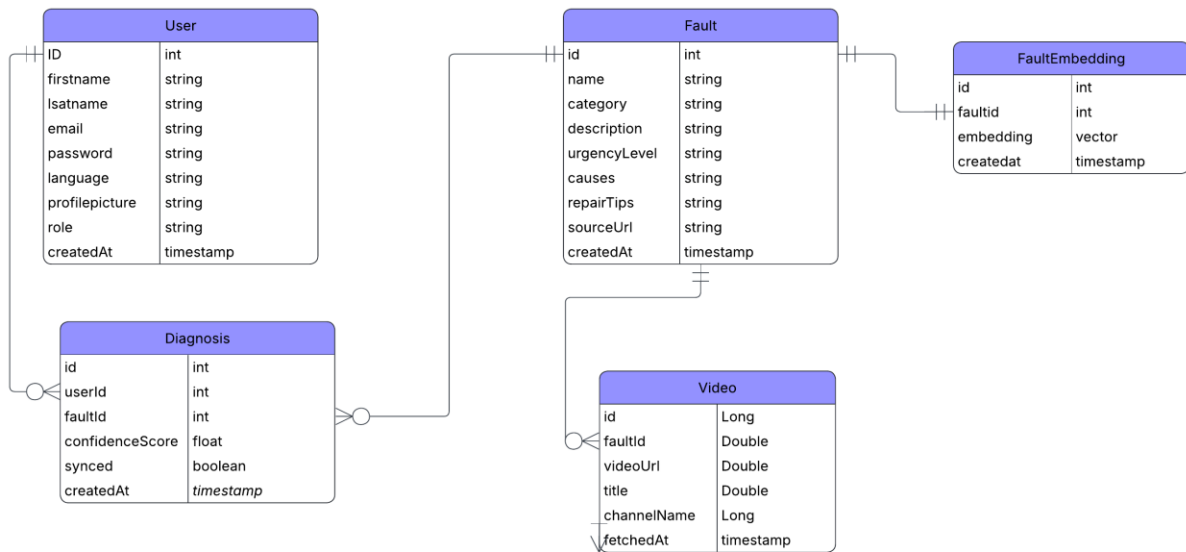
4. Entity Relationship Diagram

The entity relationship diagram below presents the five entities, their attributes, and the relationships between them. Primary keys are marked PK and foreign keys are marked FK. The notation uses crow's foot relationships, where the branched end indicates the many sides of a relationship.

The structure of the diagram is summarized as follows:

Entity	Primary Key	Foreign Keys	Connects To
USER	id (PK)	—	DIAGNOSIS (1 to many)
FAULT	id (PK)	—	FAULT_EMBEDDING (1 to 1), DIAGNOSIS (1 to many), VIDEO (1 to many)
FAULT_EMBEDDING	id (PK)	fault_id (FK)	FAULT (1 to 1)
DIAGNOSIS	id (PK)	user_id (FK), matched_fault_id (FK)	USER (many to 1), FAULT (many to 1)
VIDEO	id (PK)	fault_id (FK)	FAULT (many to 1)

Reading the diagram, the FAULT entity sits at the center of the design. It connects to FAULT_EMBEDDING through a one-to-one relationship, and to both DIAGNOSIS and VIDEO through one-to-many relationships. The USER entity connects to DIAGNOSIS through a one-to-many relationship. This structure ensures that all diagnostic activity and all supporting data revolve around the central fault catalogue.



Type something

5. Database Implementation

The database was implemented using PostgreSQL, an open-source relational database management system. PostgreSQL was selected because it is reliable, supports advanced data types, and most importantly supports the pgvector extension, which is required for storing and searching the embedding vectors used by the artificial intelligence component.

5.1 Database Management System

Property	Choice	Reason
DBMS	PostgreSQL	Reliable, open-source, supports vector search
Vector Extension	pgvector	Stores and searches embedding vectors
Hosting	Docker container	Easy and consistent local setup

5.2 Enabling the Vector Extension

Before creating the tables, the pgvector extension was enabled to allow the database to store vector data and perform similarity search:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

5.3 Table Creation

The fault catalogue table and the user, diagnosis, and cache tables were generated automatically by the backend object relational mapping framework. The embedding table, which uses the special vector type, was created with the following structure:

```
CREATE TABLE fault_embeddings (  
  id          BIGSERIAL PRIMARY KEY,  
  fault_id    VARCHAR(20) NOT NULL UNIQUE,  
  modality    VARCHAR(20),  
  source_text TEXT,  
  embedding   vector(1024) NOT NULL,  
  model_version VARCHAR(50),  
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),  
  CONSTRAINT fk_fault FOREIGN KEY (fault_id)  
    REFERENCES faults(id) ON DELETE CASCADE  
);
```

5.4 Vector Index for Fast Search

To make similarity search fast across more than six thousand fault vectors, an IVFFlat index was created on the embedding column using the cosine distance operator:

```
CREATE INDEX idx_fault_embeddings_cosine  
  ON fault_embeddings  
  USING ivfflat (embedding vector_cosine_ops)  
  WITH (lists = 100);
```

6. Backend Implementation

The backend of the CarDiag application was implemented using the Spring Boot framework with the Java programming language. The backend is responsible for receiving requests from the mobile application, processing them, communicating with the database, and returning responses.

6.1 Layered Architecture

The backend follows a clean layered architecture that separates concerns and keeps the code organized and maintainable:

Layer	Responsibility
Controller	Receives HTTP requests from the mobile application and returns responses.
Service	Contains the business logic such as the diagnosis pipeline.
Repository	Communicates directly with the database to read and write data.
Entity	Java classes that map directly to database tables.

6.2 Entity Mapping

Each database table is represented in the backend by a Java entity class. The mapping is achieved using the Java Persistence API annotations. For example, the Fault entity maps to the faults table, with each field mapping to a column. The lists of symptoms, causes, and repair tips are stored as JSON text using a custom converter, avoiding the need for additional tables.

6.3 Repositories

Repositories provide the methods used to access the database. They were implemented by extending the JpaRepository interface, which automatically provides standard operations such as saving, finding, and deleting records. Custom queries were added where needed, such as the native vector similarity query used to find the closest matching fault.

6.4 The Diagnosis Pipeline

The core of the backend is the diagnosis service, which orchestrates the entire process of turning a user input into a diagnosis. The steps are:

- The input is described in text form using an external AI model.
- The description is converted into an embedding vector.
- The vector is compared against all stored fault embeddings to find the closest match.
- If the match is confident enough, the full fault details are retrieved from the database.
- An AI report is generated and relevant video tutorials are fetched.
- The complete diagnosis is saved to the database and returned to the user.

7. Connecting the Database to the Backend

The connection between the backend and the PostgreSQL database was configured in the application properties file. This file tells the backend where the database is located and how to authenticate with it.

7.1 Connection Configuration

The following configuration establishes the database connection:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/cardiag
spring.datasource.username=postgres
```

```
spring.datasource.password=*****  
spring.datasource.driver-class-name=org.postgresql.Driver
```

7.2 Object Relational Mapping Configuration

The Java Persistence API was configured to map entities to tables and to validate the schema on startup:

```
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

7.3 How the Connection Works

When the backend starts, it reads the connection settings, establishes a connection pool to the PostgreSQL database, and verifies that the entity classes match the database tables. From that point on, every repository method call is translated into a database query, executed against PostgreSQL, and the results are mapped back into Java objects automatically.

For the vector operations, which are not part of standard object relational mapping, native SQL queries were used. The float array vectors produced by the AI model are converted into the string format that pgvector understands before being stored, and the cosine distance operator is used in the search query to find the most similar faults.

7.4 Data Population

Two one-time pipelines were used to populate the database. The first pipeline read the catalogue of six thousand one hundred and fifty-two OBD-II fault codes from a structured file and saved them into the faults table. The second pipeline read each fault, generated its embedding vector using the embedding model, and stored the vector in the fault embeddings table. Once populated, the database was ready to serve diagnosis requests.

8. Conclusion

This report has presented the complete database design and implementation for the CarDiag mobile application. The data elements were identified from the system requirements, a conceptual design was produced defining the entities and their relationships, and an entity relationship diagram was used to represent the structure. The database was implemented in PostgreSQL with the pgvector extension to support artificial intelligence similarity search, and the backend was implemented in Spring Boot following a clean layered architecture. Finally, the database was connected to the backend through configuration and object relational mapping, and populated with the full fault catalogue and its embeddings.

The resulting database provides a solid and scalable foundation that supports all the core features of the application, including fault matching, diagnosis history, video tutorials, and offline data synchronization.